

(ADMM) Model-Based Deep Learning for Robust PCA in Internet Traffic Decomposition :

From Pure Model-Based Optimization to Model-Driven Deep Learning

This notebook studies Robust Principal Component Analysis (RPCA) for internet traffic decomposition under the model:

$$[Y = L_0 + S_0 + N]$$

where:

- (L_0) = low-rank nominal traffic
- (S_0) = sparse structured anomalies
- (N) = noise
- (Y) = observed traffic matrix

The objective is not pure anomaly detection, but **accurate reconstruction of the low-rank and sparse components**, improving over classical optimization through learning-based extensions.

Implemented Approaches

We progressively move from fully model-based ADMM to deep unfolded and hybrid architectures:

- **RUN1 — ADMM (Ideal)**
Classical ADMM evaluated on the theoretical dataset (matched assumptions).
 - **RUN2 — ADMM (Baseline Mismatch)**
Same ADMM configuration evaluated under dataset mismatch conditions.
 - **RUN3 — Global-Learned ADMM**
ADMM with globally learned hyperparameters.
 - **RUN4 — Layer-wise Unfolded ADMM**
Deep unfolding of ADMM with layer-wise learnable parameters.
 - **RUN5 — Unfolded ADMM + Learned Prox**
Unfolded ADMM with a learned proximal operator for the sparse component.
 - **RUN6 — Hybrid DL Residual ADMM**
Hybrid architecture combining model-based structure with residual deep learning.
-

Evaluation Metrics

Primary reconstruction metrics:

- **NMSE_L** — Normalized Mean Squared Error on the low-rank component
- **NMSE_S** — Normalized Mean Squared Error on the sparse component

Secondary analysis:

- Convergence behavior
- Iteration efficiency
- Reconstruction stability under mismatch

This notebook highlights the transition from **pure model-based optimization** to **model-driven deep learning**, analyzing when and how learning improves structured matrix decomposition. \

```
In [1]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [16]: # Imports
import os, sys, time, math, json
import numpy as np
import matplotlib.pyplot as plt
import h5py
import torch
import torch.nn as nn
import torch.optim as optim
from tqdm import tqdm
import warnings
warnings.filterwarnings("ignore", category=SyntaxWarning)

# Project paths
PROJECT_PATH = "/content/drive/MyDrive/RPCA_Project"
UTILS_PATH = os.path.join(PROJECT_PATH, "utils")
sys.path.append(UTILS_PATH)

# Custom modules
from admm_utils import soft_threshold, admm_rpca, svt
from metrics import compute_nmse
from plotting import save_run_results
print("Custom modules imported successfully.")

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Reproducibility
torch.manual_seed(0)
np.random.seed(0)

# Results folders
METRICS_PATH = os.path.join(PROJECT_PATH, "results_metrics")
FIGURES_PATH = os.path.join(PROJECT_PATH, "results_figures")
```

```
os.makedirs(METRICS_PATH, exist_ok=True)
os.makedirs(FIGURES_PATH, exist_ok=True)
```

Custom modules imported successfully.
Using device: cuda

In [3]: *# Load mismatch dataset (used in RUN2-RUN6)*

```
BASE_PATH = "/content/drive/MyDrive/RPCA_Project"
MISM_PATH = os.path.join(BASE_PATH, "traffic_dataset_mismatch.mat")

with h5py.File(MISM_PATH, "r") as f:
    Y_train = np.array(f["Y_train"])
    L0_train = np.array(f["L0_train"])
    S0_train = np.array(f["S0_train"])
    Y_test = np.array(f["Y_test"])
    L0_test = np.array(f["L0_test"])
    S0_test = np.array(f["S0_test"])

Y_train = torch.tensor(Y_train).permute(0, 2, 1).float().to(device)
L0_train = torch.tensor(L0_train).permute(0, 2, 1).float().to(device)
S0_train = torch.tensor(S0_train).permute(0, 2, 1).float().to(device)

Y_test = torch.tensor(Y_test).permute(0, 2, 1).float().to(device)
L0_test = torch.tensor(L0_test).permute(0, 2, 1).float().to(device)
S0_test = torch.tensor(S0_test).permute(0, 2, 1).float().to(device)

num_train, N, T = Y_train.shape
num_test = Y_test.shape[0]

print("Mismatch dataset loaded")
print(f"Train: {Y_train.shape}")
print(f"Test : {Y_test.shape}")
```

Mismatch dataset loaded
Train: torch.Size([80, 100, 500])
Test : torch.Size([40, 100, 500])

RUN1 — ADMM (Ideal)

Classical ADMM is evaluated on the theoretical dataset, where the RPCA model assumptions are fully satisfied:

$$[Y = L_0 + S_0 + N]$$

No learning is involved in this run.

This experiment serves as the ideal baseline, providing a reference performance under perfectly matched model conditions.

In [4]: *# RUN1 - ADMM (Ideal)*

```
print("\n=== RUN1 - ADMM (Ideal) ===")

# Load theoretical dataset

THEORETICAL_DATA_PATH = os.path.join(PROJECT_PATH, "traffic_dataset_theoretical_full")

with h5py.File(THEORETICAL_DATA_PATH, "r") as f:
    Y_test_theo = torch.tensor(np.array(f["Y_test"])).permute(0, 2, 1).float().to(device)
    L0_test_theo = torch.tensor(np.array(f["L0_test"])).permute(0, 2, 1).float().to(device)
    S0_test_theo = torch.tensor(np.array(f["S0_test"])).permute(0, 2, 1).float().to(device)
```

```

num_test_theo = Y_test_theo.shape[0]
N_theo, T_theo = Y_test_theo.shape[1:]

print(f"Test samples: {num_test_theo}")
print(f"Matrix size : {N_theo} x {T_theo}")

# Hyperparameters

lambda_val = 1 / math.sqrt(max(N_theo, T_theo))
rho = 1.0

print(f"lambda = {lambda_val:.6f} | rho = {rho}")

NMSE_L, NMSE_S, Iterations = [], [], []

L_rep = S_rep = L0_rep = S0_rep = convergence_curve_rep = None

# ADMM evaluation loop

for i in range(num_test_theo):

    L, S, iters, conv = admm_rpca(Y_test_theo[i], lambda_val, rho)

    nmse_L = compute_nmse(L, L0_test_theo[i]).item()
    nmse_S = compute_nmse(S, S0_test_theo[i]).item()

    NMSE_L.append(nmse_L)
    NMSE_S.append(nmse_S)
    Iterations.append(iters)

    if i == 0:
        L_rep = L.detach().cpu().numpy()
        S_rep = S.detach().cpu().numpy()
        L0_rep = L0_test_theo[i].detach().cpu().numpy()
        S0_rep = S0_test_theo[i].detach().cpu().numpy()
        convergence_curve_rep = conv

    if i == 0 or (i+1) % 10 == 0:
        print(f"Test {i+1:3d}/{num_test_theo} | "
              f"NMSE_L={nmse_L:.6f} | "
              f"NMSE_S={nmse_S:.6f} | "
              f"Iter={iters}")

# Aggregate results

NMSE_L = np.array(NMSE_L)
NMSE_S = np.array(NMSE_S)
Iterations = np.array(Iterations)

print("\n==== RUN1 RESULTS ====")
print(f"NMSE_L      : {NMSE_L.mean():.6f} ± {NMSE_L.std():.6f}")
print(f"NMSE_S      : {NMSE_S.mean():.6f} ± {NMSE_S.std():.6f}")
print(f"Iterations   : {Iterations.mean():.1f} ± {Iterations.std():.1f}")
print("=====")

# Save results

save_run_results(
    run_name="RUN1_ADMM_IDEAL",
    nmse_L_list=NMSE_L,
    nmse_S_list=NMSE_S,
    iterations_list=Iterations,
    L0_rep=L0_rep,

```



```

L_hat_rep=L_rep,
S0_rep=S0_rep,
S_hat_rep=S_rep,
convergence_curve=convergence_curve_rep,
extra_params={"lambda": lambda_val, "rho": rho},
base_path=PROJECT_PATH
)

print("\nRUN1 COMPLETED.")

```

=== RUN1 – ADMM (Ideal) ===

Test samples: 40

Matrix size : 100 x 500

lambda = 0.044721 | rho = 1.0

Test 1/40	NMSE_L=0.000591	NMSE_S=0.023679	Iter=431
Test 10/40	NMSE_L=0.000578	NMSE_S=0.023520	Iter=433
Test 20/40	NMSE_L=0.000574	NMSE_S=0.023560	Iter=432
Test 30/40	NMSE_L=0.000601	NMSE_S=0.023591	Iter=438
Test 40/40	NMSE_L=0.000583	NMSE_S=0.023360	Iter=430

===== RUN1 RESULTS =====

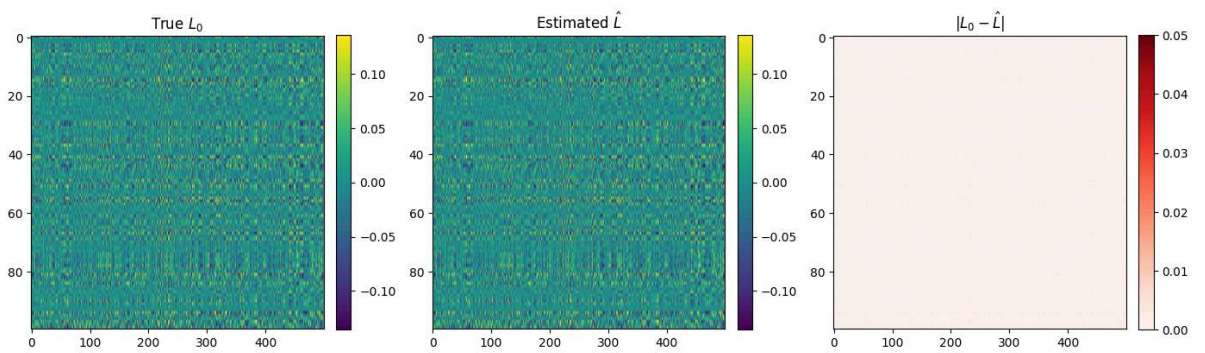
NMSE_L : 0.000580 ± 0.000012

NMSE_S : 0.023465 ± 0.000153

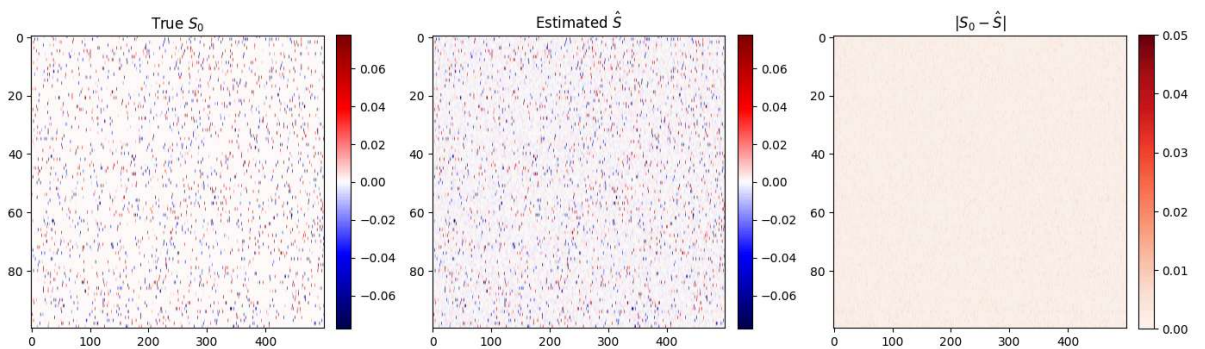
Iterations : 432.9 ± 3.5

=====

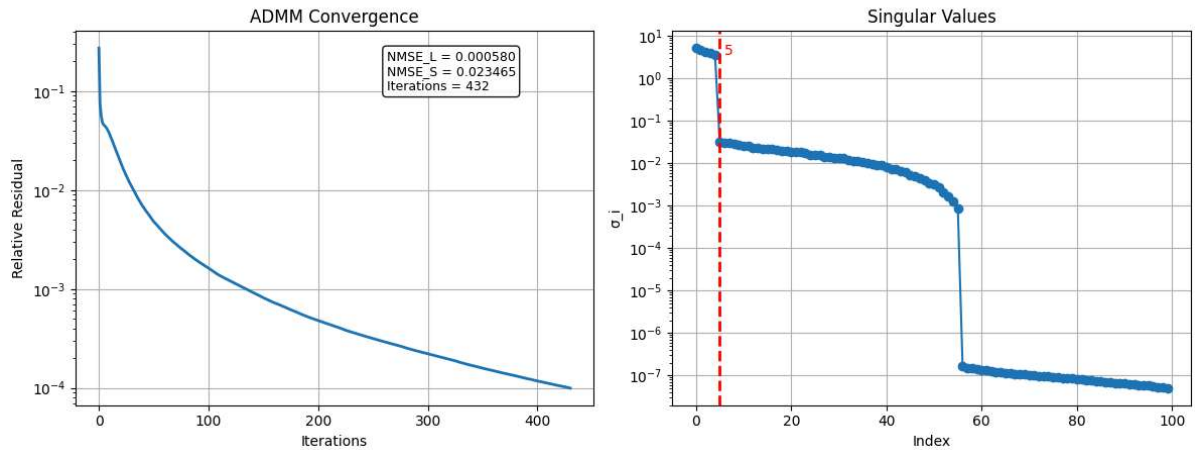
RUN1_ADMM_IDEAL — LOW RANK



RUN1_ADMM_IDEAL — SPARSE



RUN1_ADMM_IDEAL — DIAGNOSTICS



Saved structured figures for RUN1_ADMM_IDEAL

RUN1 COMPLETED.

RUN2 — ADMM (Baseline Mismatch)

Classical ADMM is evaluated on the mismatch dataset.

The RPCA model is still assumed, but the data no longer perfectly satisfy the theoretical assumptions.

No learning is involved in this run.

This experiment measures the degradation of pure model-based ADMM under model mismatch conditions.

```
In [5]: # RUN2 - ADMM (Baseline Mismatch)

print("\n=== RUN2 - ADMM (Baseline Mismatch) ===")

num_test = Y_test.shape[0]
N, T = Y_test.shape[1:]

lambda_reg = 1 / math.sqrt(max(N, T))
rho = 1.0
max_iter = 2000
tol = 1e-4

print(f"lambda = {lambda_reg:.6f} | rho = {rho}")
print("\nRunning ADMM on full test set...\n")

NMSE_L, NMSE_S, Iterations = [], [], []
L_rep = S_rep = L0_rep = S0_rep = convergence_curve_rep = None

for i in range(num_test):
    L, S, iters, conv = admm_rpca(
        Y_test[i],
        lambda_reg,
        rho,
        max_iter=max_iter,
        tol=tol
    )

    nmse_L = compute_nmse(L, L0_test[i]).item()
    nmse_S = compute_nmse(S, S0_test[i]).item()
```

```

NMSE_L.append(nmse_L)
NMSE_S.append(nmse_S)
Iterations.append(iters)

if i == 0:
    L_rep = L.detach().cpu().numpy()
    S_rep = S.detach().cpu().numpy()
    L0_rep = L0_test[i].detach().cpu().numpy()
    S0_rep = S0_test[i].detach().cpu().numpy()
    convergence_curve_rep = conv

if i == 0 or (i+1) % 10 == 0:
    print(f"Test {i+1:3d}/{num_test} | "
          f"NMSE_L={nmse_L:.6f} | "
          f"NMSE_S={nmse_S:.6f} | "
          f"Iter={iters}")

NMSE_L = np.array(NMSE_L)
NMSE_S = np.array(NMSE_S)
Iterations = np.array(Iterations)

print("\n==== RUN2 RESULTS =====")
print(f"NMSE_L      : {NMSE_L.mean():.6f} ± {NMSE_L.std():.6f}")
print(f"NMSE_S      : {NMSE_S.mean():.6f} ± {NMSE_S.std():.6f}")
print(f"Iterations   : {Iterations.mean():.1f} ± {Iterations.std():.1f}")
print("=====")

save_run_results(
    run_name="RUN2_ADMM_BASELINE_MISMATCH",
    nmse_L_list=NMSE_L,
    nmse_S_list=NMSE_S,
    iterations_list=Iterations,
    L0_rep=L0_rep,
    L_hat_rep=L_rep,
    S0_rep=S0_rep,
    S_hat_rep=S_rep,
    convergence_curve=convergence_curve_rep,
    layerwise_nmse_L=None,
    layerwise_nmse_S=None,
    extra_params={
        "lambda": lambda_reg,
        "rho": rho,
        "max_iter": max_iter,
        "tol": tol
    },
    base_path=PROJECT_PATH
)

print("\nRUN2 COMPLETED.")

```

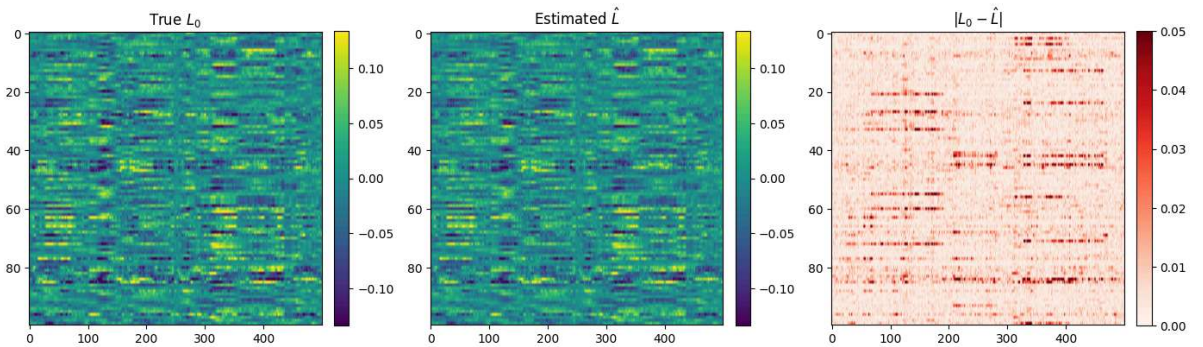
=== RUN2 — ADMM (Baseline Mismatch) ===
 $\lambda = 0.044721$ | $\rho = 1.0$

Running ADMM on full test set...

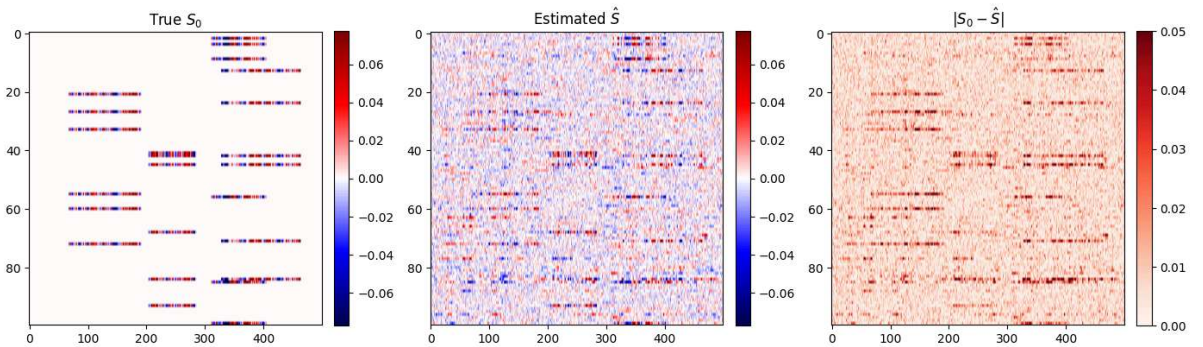
Test	1/40	NMSE_L=0.050149	NMSE_S=0.778272	Iter=189
Test	10/40	NMSE_L=0.053602	NMSE_S=0.819946	Iter=188
Test	20/40	NMSE_L=0.062310	NMSE_S=0.921664	Iter=187
Test	30/40	NMSE_L=0.056516	NMSE_S=0.856687	Iter=183
Test	40/40	NMSE_L=0.060494	NMSE_S=0.897079	Iter=185

===== RUN2 RESULTS =====
 NMSE_L : 0.053764 ± 0.005426
 NMSE_S : 0.822088 ± 0.057970
 Iterations : 187.8 ± 2.6
 =====

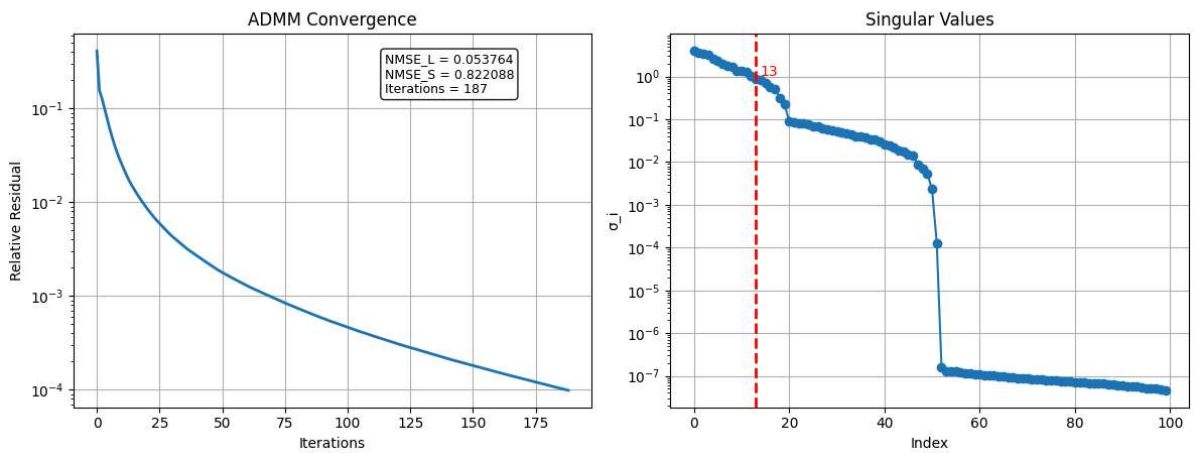
RUN2_ADMM_BASELINE_MISMATCH — LOW RANK



RUN2_ADMM_BASELINE_MISMATCH — SPARSE



RUN2_ADMM_BASELINE_MISMATCH — DIAGNOSTICS



Saved structured figures for RUN2_ADMM_BASELINE_MISMATCH

RUN2 COMPLETED.

RUN3 — Global-Learned ADMM

ADMM hyperparameters are selected using a grid search on a subset of the training data.

The optimal global values are chosen by minimizing a joint reconstruction score based on both low-rank and sparse NMSE.

The selected hyperparameters are then evaluated on the full test set.

No unfolding or learned operators are introduced yet.

This run improves classical ADMM through data-driven hyperparameter selection.

```
In [6]: # RUN3 - Global-Learned ADMM

print("\n=== RUN3 - Global-Learned ADMM ===")

num_train = Y_train.shape[0]
num_test  = Y_test.shape[0]
N, T      = Y_train.shape[1:]

max_iter_train, tol_train = 300, 1e-3
max_iter_test,  tol_test  = 2000, 1e-4

# Grid search on training subset

print("\n--- Grid search (train) ---")

lambda0 = 1 / math.sqrt(max(N, T))
lambda_grid = lambda0 * np.array([0.25, 0.5, 0.75, 1, 1.25, 1.5, 2])
rho_grid = np.array([0.5, 0.75, 1.0, 1.25, 1.5, 2.0])

train_subset = min(25, num_train)

best_score = float("inf")
lambda_best = rho_best = None

for lambda_reg in lambda_grid:
    for rho in rho_grid:

        scores = []

        for idx in range(train_subset):

            L, S, _, _ = admm_rpca(
                Y_train[idx],
                lambda_reg,
                rho,
                max_iter=max_iter_train,
                tol=tol_train
            )

            nmse_L = compute_nmse(L, L0_train[idx]).item()
            nmse_S = compute_nmse(S, S0_train[idx]).item()
            scores.append(math.sqrt(nmse_L * nmse_S))

        mean_score = np.mean(scores)

        print(f"lambda={lambda_reg:.5f} | rho={rho:.2f} | score={mean_score:.6f}")

        if mean_score < best_score:
            best_score = mean_score
            lambda_best = float(lambda_reg)
            rho_best = float(rho)
```



```

print("\nBest hyperparameters (train)")
print(f"lambda_best = {lambda_best:.6f}")
print(f"rho_best    = {rho_best:.6f}")
print(f"score        = {best_score:.6f}")

# Test with Learned hyperparameters

print("\n--- Testing with learned hyperparameters ---")

NMSE_L, NMSE_S, Iterations = [], [], []
L_rep = S_rep = L0_rep = S0_rep = convergence_curve_rep = None

for i in range(num_test):

    L, S, iters, conv = admm_rpca(
        Y_test[i],
        lambda_best,
        rho_best,
        max_iter=max_iter_test,
        tol=tol_test
    )

    nmse_L = compute_nmse(L, L0_test[i]).item()
    nmse_S = compute_nmse(S, S0_test[i]).item()

    NMSE_L.append(nmse_L)
    NMSE_S.append(nmse_S)
    Iterations.append(iters)

    if i == 0:
        L_rep = L.detach().cpu().numpy()
        S_rep = S.detach().cpu().numpy()
        L0_rep = L0_test[i].detach().cpu().numpy()
        S0_rep = S0_test[i].detach().cpu().numpy()
        convergence_curve_rep = conv

    if i == 0 or (i+1) % 10 == 0:
        print(f"Test {i+1:3d}/{num_test} | "
              f"NMSE_L={nmse_L:.6f} | "
              f"NMSE_S={nmse_S:.6f} | "
              f"Iter={iters}")

# Aggregate results

NMSE_L = np.array(NMSE_L)
NMSE_S = np.array(NMSE_S)
Iterations = np.array(Iterations)

print("\n===== RUN3 RESULTS =====")
print(f"NMSE_L      : {NMSE_L.mean():.6f} ± {NMSE_L.std():.6f}")
print(f"NMSE_S      : {NMSE_S.mean():.6f} ± {NMSE_S.std():.6f}")
print(f"Iterations  : {Iterations.mean():.1f} ± {Iterations.std():.1f}")
print("=====")

# Save results

save_run_results(
    run_name="RUN3_GLOBAL_LEARNED_ADMM",
    nmse_L_list=NMSE_L,

```

```
nmse_S_list=NMSE_S,  
iterations_list=Iterations,  
L0_rep=L0_rep,  
L_hat_rep=L_rep,  
S0_rep=S0_rep,  
S_hat_rep=S_rep,  
convergence_curve=convergence_curve_rep,  
extra_params={  
    "lambda_best": lambda_best,  
    "rho_best": rho_best,  
    "train_score": best_score  
},  
base_path=PROJECT_PATH  
)  
  
print("\nRUN3 COMPLETED.")
```

=== RUN3 – Global-Learned ADMM ===

--- Grid search (train) ---

lambda=0.01118	rho=0.50	score=3.399378
lambda=0.01118	rho=0.75	score=3.399378
lambda=0.01118	rho=1.00	score=3.399378
lambda=0.01118	rho=1.25	score=3.399378
lambda=0.01118	rho=1.50	score=3.399378
lambda=0.01118	rho=2.00	score=3.399378
lambda=0.02236	rho=0.50	score=1.654916
lambda=0.02236	rho=0.75	score=1.662906
lambda=0.02236	rho=1.00	score=1.646594
lambda=0.02236	rho=1.25	score=1.656474
lambda=0.02236	rho=1.50	score=1.674456
lambda=0.02236	rho=2.00	score=1.673661
lambda=0.03354	rho=0.50	score=0.349322
lambda=0.03354	rho=0.75	score=0.349506
lambda=0.03354	rho=1.00	score=0.349836
lambda=0.03354	rho=1.25	score=0.350123
lambda=0.03354	rho=1.50	score=0.351014
lambda=0.03354	rho=2.00	score=0.352876
lambda=0.04472	rho=0.50	score=0.209296
lambda=0.04472	rho=0.75	score=0.209301
lambda=0.04472	rho=1.00	score=0.209300
lambda=0.04472	rho=1.25	score=0.209293
lambda=0.04472	rho=1.50	score=0.209343
lambda=0.04472	rho=2.00	score=0.208916
lambda=0.05590	rho=0.50	score=0.242116
lambda=0.05590	rho=0.75	score=0.242113
lambda=0.05590	rho=1.00	score=0.242103
lambda=0.05590	rho=1.25	score=0.242082
lambda=0.05590	rho=1.50	score=0.242084
lambda=0.05590	rho=2.00	score=0.241646
lambda=0.06708	rho=0.50	score=0.289867
lambda=0.06708	rho=0.75	score=0.289979
lambda=0.06708	rho=1.00	score=0.290104
lambda=0.06708	rho=1.25	score=0.290035
lambda=0.06708	rho=1.50	score=0.289833
lambda=0.06708	rho=2.00	score=0.289823
lambda=0.08944	rho=0.50	score=0.339834
lambda=0.08944	rho=0.75	score=0.339970
lambda=0.08944	rho=1.00	score=0.340055
lambda=0.08944	rho=1.25	score=0.340134
lambda=0.08944	rho=1.50	score=0.340069
lambda=0.08944	rho=2.00	score=0.339953

Best hyperparameters (train)

lambda_best = 0.044721
rho_best = 2.000000
score = 0.208916

--- Testing with learned hyperparameters ---

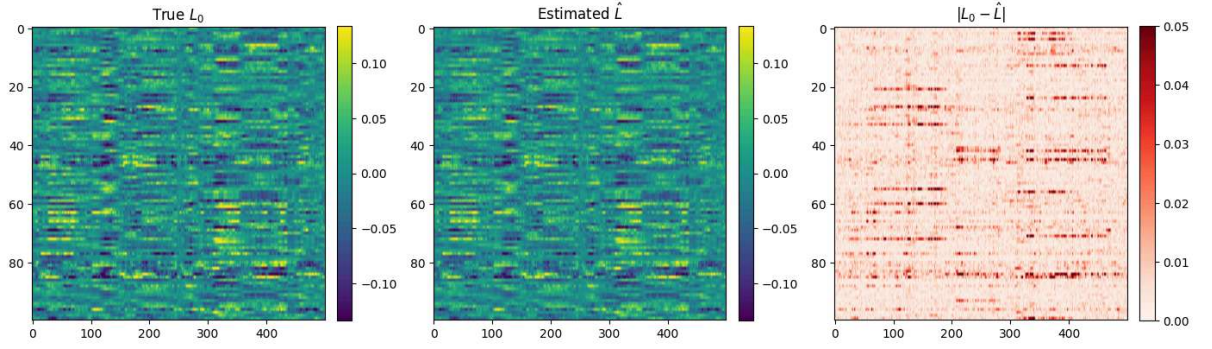
Test 1/40	NMSE_L=0.050148	NMSE_S=0.778269	Iter=96
Test 10/40	NMSE_L=0.053598	NMSE_S=0.819912	Iter=95
Test 20/40	NMSE_L=0.062308	NMSE_S=0.921636	Iter=95
Test 30/40	NMSE_L=0.056514	NMSE_S=0.856667	Iter=93
Test 40/40	NMSE_L=0.060491	NMSE_S=0.897051	Iter=94

===== RUN3 RESULTS =====

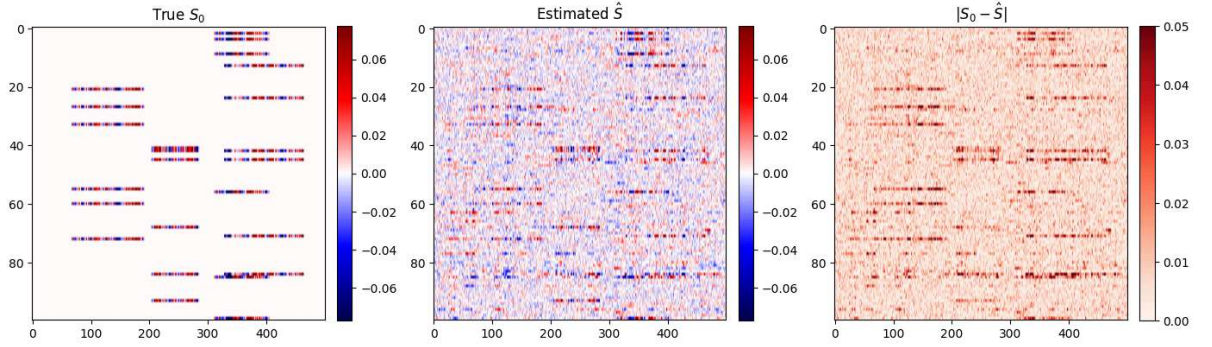
NMSE_L : 0.053762 ± 0.005426
NMSE_S : 0.822068 ± 0.057970
Iterations : 94.9 ± 1.4

=====

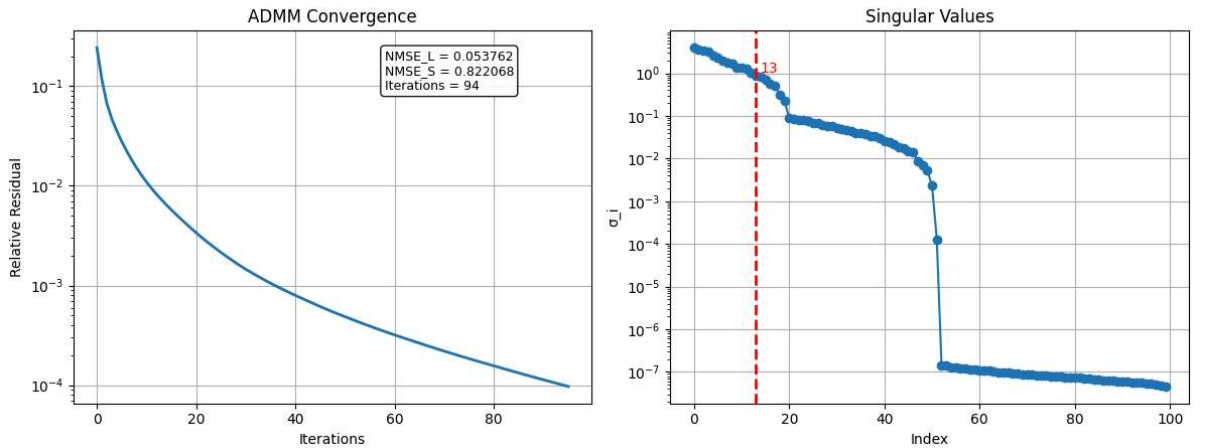
RUN3_GLOBAL_LEARNED_ADMM — LOW RANK



RUN3_GLOBAL_LEARNED_ADMM — SPARSE



RUN3_GLOBAL_LEARNED_ADMM — DIAGNOSTICS



Saved structured figures for RUN3_GLOBAL_LEARNED_ADMM

RUN3 COMPLETED.

RUN4 — Layer-wise Unfolded ADMM

ADMM is unfolded into a finite-depth architecture with K layers.

Each layer has learnable ADMM hyperparameters λ_k and $\rho_{k'}$, which are optimized through backpropagation on the training set.

The algorithmic structure of ADMM is preserved, but the parameters become layer-dependent and data-driven.

This run introduces model-based deep learning through unfolding, while keeping the original proximal operators unchanged.

In [7]: *# RUN4 – Layer-wise Unfolded ADMM*

```

print("\n=== RUN4 - Layer-wise Unfolded ADMM ===")

# Model

class UnfoldedADMM(nn.Module):
    def __init__(self, K, lambda_init, rho_init):
        super().__init__()
        self.K = K
        self.log_lambda = nn.Parameter(torch.log(torch.ones(K) * lambda_init))
        self.log_rho = nn.Parameter(torch.log(torch.ones(K) * rho_init))

    def forward(self, Y, L0=None, S0=None, return_curve=False):

        L = torch.zeros_like(Y)
        S = torch.zeros_like(Y)
        U = torch.zeros_like(Y)

        nmse_L_curve, nmse_S_curve = [], []

        for k in range(self.K):

            lambda_k = torch.exp(self.log_lambda[k])
            rho_k = torch.exp(self.log_rho[k])

            # ADMM updates
            L = svt(Y - S + U, 1.0 / rho_k)
            S = soft_threshold(Y - L + U, lambda_k / rho_k)
            U = U + (Y - L - S)

            # Optional layer-wise NMSE tracking
            if return_curve and (L0 is not None) and (S0 is not None):
                nmse_L_curve.append(compute_nmse(L, L0).item())
                nmse_S_curve.append(compute_nmse(S, S0).item())

        if return_curve:
            return L, S, nmse_L_curve, nmse_S_curve
        return L, S

# Hyperparameters

K = 10
epochs = 100
batch_size = 4
lr = 5e-4
patience = 10

lambda_init = 1 / math.sqrt(max(N, T))
rho_init = 2.0

model = UnfoldedADMM(K, lambda_init, rho_init).to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

# Train / validation split

val_size = 10
train_idx = list(range(val_size, num_train))
val_idx = list(range(val_size))

best_val_loss = float("inf")
patience_counter = 0

```

```

print(f"Layers: {K}")
print("Training...\n")

# Training Loop

for epoch in range(epochs):

    model.train()
    train_loss = 0.0
    perm = torch.randperm(len(train_idx))

    for i in range(0, len(train_idx), batch_size):

        batch_ids = [train_idx[j.item()] for j in perm[i:i+batch_size]]

        Y = Y_train[batch_ids]
        L0 = L0_train[batch_ids]
        S0 = S0_train[batch_ids]

        L_hat, S_hat = model(Y)
        loss = compute_nmse(L_hat, L0) + compute_nmse(S_hat, S0)

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()

        train_loss += loss.item()

    train_loss /= (len(train_idx) / batch_size)

# Validation

model.eval()
val_loss = 0.0

with torch.no_grad():
    for i in val_idx:
        Y = Y_train[i:i+1]
        L0 = L0_train[i:i+1]
        S0 = S0_train[i:i+1]

        L_hat, S_hat = model(Y)
        val_loss += (compute_nmse(L_hat, L0) +
                     compute_nmse(S_hat, S0)).item()

val_loss /= len(val_idx)

print(f"Epoch {epoch+1:3d} | Train {train_loss:.6f} | Val {val_loss:.6f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = model.state_dict()
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

# Restore best model

```

```

model.load_state_dict(best_state)

# Test

print("\nTesting RUN4...\n")

model.eval()

with torch.no_grad():

    L_hat, S_hat = model(Y_test)
    nmse_L = compute_nmse(L_hat, L0_test).item()
    nmse_S = compute_nmse(S_hat, S0_test).item()

    # Layer-wise NMSE curve (first test sample)
    _, _, nmse_L_curve, nmse_S_curve = model(
        Y_test[0:1],
        L0_test[0:1],
        S0_test[0:1],
        return_curve=True
    )

print("==== RUN4 FINAL RESULTS ====")
print(f"NMSE_L : {nmse_L:.6f}")
print(f"NMSE_S : {nmse_S:.6f}")
print(f"Layers : {K}")
print("=====")

# Save results

L_rep = L_hat[0].detach().cpu().numpy()
S_rep = S_hat[0].detach().cpu().numpy()
L0_rep = L0_test[0].detach().cpu().numpy()
S0_rep = S0_test[0].detach().cpu().numpy()

save_run_results(
    run_name="RUN4_LAYERWISE_UNFOLDED_ADM",
    nmse_L_list=[nmse_L],
    nmse_S_list=[nmse_S],
    iterations_list=[K],
    L0_rep=L0_rep,
    L_hat_rep=L_rep,
    S0_rep=S0_rep,
    S_hat_rep=S_rep,
    convergence_curve=None,
    layerwise_nmse_L=nmse_L_curve,
    layerwise_nmse_S=nmse_S_curve,
    extra_params={
        "K": K,
        "epochs": epoch+1,
        "lambda_init": lambda_init,
        "rho_init": rho_init
    },
    base_path=PROJECT_PATH
)

print("\nRUN4 COMPLETED.")

```

=== RUN4 – Layer-wise Unfolded ADMM ===

Layers: 10

Training...

Epoch	1	Train	0.945575	Val	0.929621
Epoch	2	Train	0.910235	Val	0.895879
Epoch	3	Train	0.875942	Val	0.861463
Epoch	4	Train	0.841555	Val	0.827852
Epoch	5	Train	0.806811	Val	0.796594
Epoch	6	Train	0.776957	Val	0.768460
Epoch	7	Train	0.749756	Val	0.743690
Epoch	8	Train	0.725458	Val	0.721984
Epoch	9	Train	0.704234	Val	0.702955
Epoch	10	Train	0.686736	Val	0.686266
Epoch	11	Train	0.670991	Val	0.671608
Epoch	12	Train	0.655939	Val	0.658635
Epoch	13	Train	0.642816	Val	0.647043
Epoch	14	Train	0.632394	Val	0.636642
Epoch	15	Train	0.623132	Val	0.627248
Epoch	16	Train	0.612456	Val	0.618715
Epoch	17	Train	0.601498	Val	0.610938
Epoch	18	Train	0.598011	Val	0.603832
Epoch	19	Train	0.587505	Val	0.597271
Epoch	20	Train	0.582583	Val	0.591198
Epoch	21	Train	0.577401	Val	0.585555
Epoch	22	Train	0.570510	Val	0.580263
Epoch	23	Train	0.566331	Val	0.575353
Epoch	24	Train	0.560273	Val	0.570695
Epoch	25	Train	0.557896	Val	0.566320
Epoch	26	Train	0.553578	Val	0.562233
Epoch	27	Train	0.548456	Val	0.558387
Epoch	28	Train	0.543792	Val	0.554726
Epoch	29	Train	0.540239	Val	0.551218
Epoch	30	Train	0.536381	Val	0.547883
Epoch	31	Train	0.532864	Val	0.544673
Epoch	32	Train	0.530499	Val	0.541641
Epoch	33	Train	0.527973	Val	0.538704
Epoch	34	Train	0.526829	Val	0.535873
Epoch	35	Train	0.523655	Val	0.533159
Epoch	36	Train	0.521048	Val	0.530545
Epoch	37	Train	0.515941	Val	0.528042
Epoch	38	Train	0.516329	Val	0.525637
Epoch	39	Train	0.513920	Val	0.523328
Epoch	40	Train	0.509519	Val	0.521088
Epoch	41	Train	0.507921	Val	0.518937
Epoch	42	Train	0.503619	Val	0.516899
Epoch	43	Train	0.504685	Val	0.514928
Epoch	44	Train	0.504803	Val	0.513031
Epoch	45	Train	0.499675	Val	0.511233
Epoch	46	Train	0.499678	Val	0.509495
Epoch	47	Train	0.496901	Val	0.507820
Epoch	48	Train	0.494668	Val	0.506234
Epoch	49	Train	0.493527	Val	0.504735
Epoch	50	Train	0.492377	Val	0.503293
Epoch	51	Train	0.488994	Val	0.501900
Epoch	52	Train	0.487137	Val	0.500582
Epoch	53	Train	0.488031	Val	0.499310
Epoch	54	Train	0.486702	Val	0.498106
Epoch	55	Train	0.483676	Val	0.496922
Epoch	56	Train	0.484873	Val	0.495790
Epoch	57	Train	0.482946	Val	0.494712
Epoch	58	Train	0.482336	Val	0.493671
Epoch	59	Train	0.481287	Val	0.492651
Epoch	60	Train	0.478611	Val	0.491707

Epoch 61	Train 0.478989	Val 0.490805
Epoch 62	Train 0.477987	Val 0.489939
Epoch 63	Train 0.478862	Val 0.489107
Epoch 64	Train 0.474186	Val 0.488328
Epoch 65	Train 0.477137	Val 0.487555
Epoch 66	Train 0.475844	Val 0.486822
Epoch 67	Train 0.474211	Val 0.486084
Epoch 68	Train 0.473042	Val 0.485379
Epoch 69	Train 0.471854	Val 0.484691
Epoch 70	Train 0.471530	Val 0.484037
Epoch 71	Train 0.472868	Val 0.483391
Epoch 72	Train 0.472006	Val 0.482781
Epoch 73	Train 0.470707	Val 0.482175
Epoch 74	Train 0.469653	Val 0.481581
Epoch 75	Train 0.469869	Val 0.480988
Epoch 76	Train 0.469862	Val 0.480430
Epoch 77	Train 0.469773	Val 0.479887
Epoch 78	Train 0.467921	Val 0.479362
Epoch 79	Train 0.468700	Val 0.478839
Epoch 80	Train 0.465378	Val 0.478330
Epoch 81	Train 0.467957	Val 0.477830
Epoch 82	Train 0.465182	Val 0.477339
Epoch 83	Train 0.465620	Val 0.476861
Epoch 84	Train 0.465559	Val 0.476386
Epoch 85	Train 0.466102	Val 0.475929
Epoch 86	Train 0.467128	Val 0.475476
Epoch 87	Train 0.462687	Val 0.475057
Epoch 88	Train 0.462998	Val 0.474620
Epoch 89	Train 0.466615	Val 0.474167
Epoch 90	Train 0.461485	Val 0.473756
Epoch 91	Train 0.462573	Val 0.473342
Epoch 92	Train 0.463553	Val 0.472938
Epoch 93	Train 0.460815	Val 0.472529
Epoch 94	Train 0.460042	Val 0.472150
Epoch 95	Train 0.459677	Val 0.471754
Epoch 96	Train 0.461839	Val 0.471382
Epoch 97	Train 0.459883	Val 0.471011
Epoch 98	Train 0.459302	Val 0.470668
Epoch 99	Train 0.460188	Val 0.470311
Epoch 100	Train 0.460698	Val 0.469969

Testing RUN4...

===== RUN4 FINAL RESULTS =====

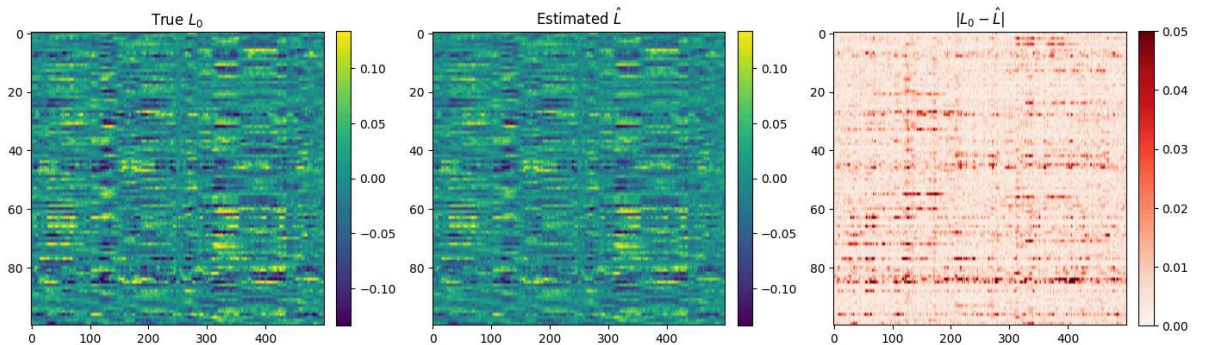
NMSE_L : 0.052066

NMSE_S : 0.425005

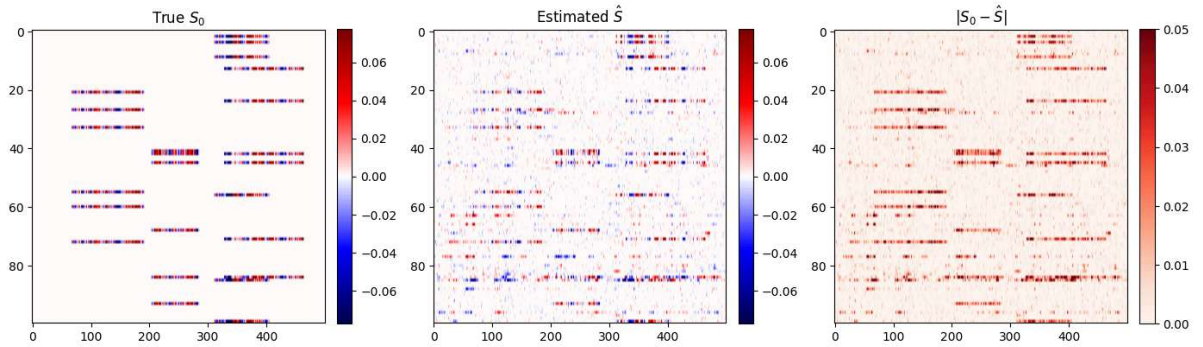
Layers : 10

=====

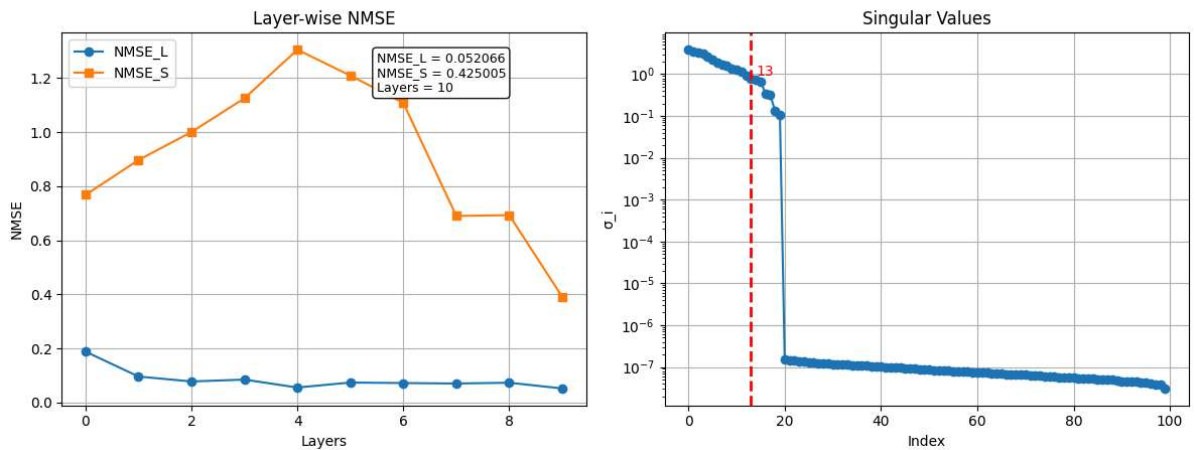
RUN4_LAYERWISE_UNFOLDED_ADMM — LOW RANK



RUN4_LAYERWISE_UNFOLDED_ADMM — SPARSE



RUN4_LAYERWISE_UNFOLDED_ADMM — DIAGNOSTICS



Saved structured figures for RUN4_LAYERWISE_UNFOLDED_ADMM

RUN4 COMPLETED.

RUN5 — Unfolded ADMM + Learned Prox

ADMM is unfolded into K layers as in RUN4, but the sparse proximal operator is replaced by a learnable neural network.

The algorithmic structure of ADMM is preserved, while the sparse update becomes data-driven.

This introduces a stronger form of model-based deep learning, allowing the network to compensate for model mismatch through a learned proximal mapping.

```
In [8]: # RUN5 — Unfolded ADMM + Learned Prox

print("\n=== RUN5 — Unfolded ADMM + Learned Prox ===")

# Sparse proximal network

class SparseProx(nn.Module):
    def __init__(self, channels=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, channels, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(channels, channels, 3, padding=1),
            nn.ReLU(),
            nn.Conv2d(channels, 1, 3, padding=1)
        )
```



```

        self.alpha = nn.Parameter(torch.tensor(0.1))

    def forward(self, Z):
        correction = self.net(Z)
        return Z - self.alpha * correction

# Deep unfolded ADMM with Learned prox

class DeepUnfoldedADMM(nn.Module):
    def __init__(self, K, lambda_init, rho_init):
        super().__init__()
        self.K = K
        self.log_lambda = nn.Parameter(torch.log(torch.ones(K) * lambda_init))
        self.log_rho = nn.Parameter(torch.log(torch.ones(K) * rho_init))
        self.prox_layers = nn.ModuleList([SparseProx(32) for _ in range(K)])

    def forward(self, Y, L0=None, S0=None, return_curve=False):

        L = torch.zeros_like(Y)
        S = torch.zeros_like(Y)
        U = torch.zeros_like(Y)

        nmse_L_curve, nmse_S_curve = [], []

        for k in range(self.K):

            lambda_k = torch.exp(self.log_lambda[k])
            rho_k = torch.exp(self.log_rho[k])

            # ADMM updates
            L = svt(Y - S + U, 1.0 / rho_k)

            Z = Y - L + U
            S = self.prox_layers[k](Z.unsqueeze(1)).squeeze(1)

            U = U + (Y - L - S)

            # Optional layer-wise NMSE tracking
            if return_curve and (L0 is not None) and (S0 is not None):
                nmse_L_curve.append(compute_nmse(L, L0).item())
                nmse_S_curve.append(compute_nmse(S, S0).item())

        if return_curve:
            return L, S, nmse_L_curve, nmse_S_curve
        return L, S

# Hyperparameters

K = 10
epochs = 100
batch_size = 4
lr = 5e-4
patience = 10

lambda_init = 1 / math.sqrt(max(N, T))
rho_init = 2.0

model = DeepUnfoldedADMM(K, lambda_init, rho_init).to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

# Train / validation split

```

```

val_size = 10
train_idx = list(range(val_size, num_train))
val_idx = list(range(val_size))

best_val_loss = float("inf")
patience_counter = 0

print(f"Layers: {K}")
print("Training...\n")

# Training Loop

for epoch in range(epochs):

    model.train()
    train_loss = 0.0
    perm = torch.randperm(len(train_idx))

    for i in range(0, len(train_idx), batch_size):

        batch_ids = [train_idx[j.item()] for j in perm[i:i+batch_size]]

        Y = Y_train[batch_ids]
        L0 = L0_train[batch_ids]
        S0 = S0_train[batch_ids]

        L_hat, S_hat = model(Y)
        loss = compute_nmse(L_hat, L0) + compute_nmse(S_hat, S0)

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()

        train_loss += loss.item()

    train_loss /= (len(train_idx) / batch_size)

# Validation

model.eval()
val_loss = 0.0

with torch.no_grad():
    for i in val_idx:
        Y = Y_train[i:i+1]
        L0 = L0_train[i:i+1]
        S0 = S0_train[i:i+1]

        L_hat, S_hat = model(Y)
        val_loss += (compute_nmse(L_hat, L0) +
                     compute_nmse(S_hat, S0)).item()

val_loss /= len(val_idx)

print(f"Epoch {epoch+1:3d} | Train {train_loss:.6f} | Val {val_loss:.6f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = model.state_dict()
    patience_counter = 0
else:

```

```

        patience_counter += 1
    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

# Restore best model

model.load_state_dict(best_state)

# Test

print("\nTesting RUN5...\n")

model.eval()

with torch.no_grad():

    L_hat, S_hat = model(Y_test)
    nmse_L = compute_nmse(L_hat, L0_test).item()
    nmse_S = compute_nmse(S_hat, S0_test).item()

    # Layer-wise NMSE curve (first test sample)
    _, _, nmse_L_curve, nmse_S_curve = model(
        Y_test[0:1],
        L0_test[0:1],
        S0_test[0:1],
        return_curve=True
    )

print("==== RUN5 FINAL RESULTS =====")
print(f"NMSE_L : {nmse_L:.6f}")
print(f"NMSE_S : {nmse_S:.6f}")
print(f"Layers : {K}")
print("=====")

# Save results

L_rep = L_hat[0].detach().cpu().numpy()
S_rep = S_hat[0].detach().cpu().numpy()
L0_rep = L0_test[0].detach().cpu().numpy()
S0_rep = S0_test[0].detach().cpu().numpy()

save_run_results(
    run_name="RUN5_UNFOLDED_ADMM_LEARNED_PROX",
    nmse_L_list=[nmse_L],
    nmse_S_list=[nmse_S],
    iterations_list=[K],
    L0_rep=L0_rep,
    L_hat_rep=L_rep,
    S0_rep=S0_rep,
    S_hat_rep=S_rep,
    convergence_curve=None,
    layerwise_nmse_L=nmse_L_curve,
    layerwise_nmse_S=nmse_S_curve,
    extra_params={
        "K": K,
        "epochs": epoch+1,
        "lambda_init": lambda_init,
        "rho_init": rho_init
    },
    base_path=PROJECT_PATH

```

```
)
```

```
print("\nRUN5 COMPLETED.")
```

=== RUN5 – Unfolded ADMM + Learned Prox ===

Layers: 10

Training...

Epoch	1	Train	11.955330	Val	10.569512
Epoch	2	Train	10.079240	Val	8.864977
Epoch	3	Train	7.872359	Val	6.166880
Epoch	4	Train	4.887395	Val	3.414031
Epoch	5	Train	2.936295	Val	2.558659
Epoch	6	Train	2.508949	Val	2.380529
Epoch	7	Train	2.408358	Val	2.289806
Epoch	8	Train	2.309645	Val	2.189310
Epoch	9	Train	2.048335	Val	1.684825
Epoch	10	Train	1.601139	Val	1.453659
Epoch	11	Train	1.423541	Val	1.303170
Epoch	12	Train	1.293291	Val	1.191193
Epoch	13	Train	1.188457	Val	1.103671
Epoch	14	Train	1.101737	Val	1.020315
Epoch	15	Train	1.029312	Val	0.958249
Epoch	16	Train	0.961789	Val	0.900915
Epoch	17	Train	0.902257	Val	0.842923
Epoch	18	Train	0.853218	Val	0.821113
Epoch	19	Train	0.821551	Val	0.778463
Epoch	20	Train	0.827289	Val	0.761841
Epoch	21	Train	0.773479	Val	0.728072
Epoch	22	Train	0.742210	Val	0.753983
Epoch	23	Train	0.761193	Val	0.746876
Epoch	24	Train	0.716310	Val	0.679779
Epoch	25	Train	0.682622	Val	0.661598
Epoch	26	Train	0.665753	Val	0.704689
Epoch	27	Train	0.682335	Val	0.660459
Epoch	28	Train	0.637091	Val	0.622694
Epoch	29	Train	0.617295	Val	0.622233
Epoch	30	Train	0.602324	Val	0.585394
Epoch	31	Train	0.578274	Val	0.572038
Epoch	32	Train	0.561951	Val	0.593615
Epoch	33	Train	0.554690	Val	0.566917
Epoch	34	Train	0.544986	Val	0.540911
Epoch	35	Train	0.556258	Val	0.635758
Epoch	36	Train	0.527978	Val	0.532739
Epoch	37	Train	0.513867	Val	0.537737
Epoch	38	Train	0.523696	Val	0.529565
Epoch	39	Train	0.515336	Val	0.505803
Epoch	40	Train	0.508212	Val	0.509188
Epoch	41	Train	0.508163	Val	0.493396
Epoch	42	Train	0.482420	Val	0.515996
Epoch	43	Train	0.499514	Val	0.481798
Epoch	44	Train	0.486995	Val	0.505989
Epoch	45	Train	0.478532	Val	0.507006
Epoch	46	Train	0.458513	Val	0.469893
Epoch	47	Train	0.451239	Val	0.480316
Epoch	48	Train	0.448037	Val	0.459235
Epoch	49	Train	0.440136	Val	0.477812
Epoch	50	Train	0.452924	Val	0.504753
Epoch	51	Train	0.456556	Val	0.463330
Epoch	52	Train	0.426334	Val	0.455605
Epoch	53	Train	0.443178	Val	0.444399
Epoch	54	Train	0.445416	Val	0.447400
Epoch	55	Train	0.430705	Val	0.456329
Epoch	56	Train	0.414538	Val	0.446855
Epoch	57	Train	0.409547	Val	0.436590
Epoch	58	Train	0.414783	Val	0.437834
Epoch	59	Train	0.408940	Val	0.421692
Epoch	60	Train	0.417897	Val	0.444301

Epoch 61	Train 0.424690	Val 0.522630
Epoch 62	Train 0.449106	Val 0.430185
Epoch 63	Train 0.407214	Val 0.444090
Epoch 64	Train 0.416334	Val 0.469764
Epoch 65	Train 0.433338	Val 0.425846
Epoch 66	Train 0.431444	Val 0.441162
Epoch 67	Train 0.424809	Val 0.418699
Epoch 68	Train 0.395426	Val 0.408300
Epoch 69	Train 0.389305	Val 0.410261
Epoch 70	Train 0.376429	Val 0.398221
Epoch 71	Train 0.387777	Val 0.426733
Epoch 72	Train 0.419285	Val 0.435960
Epoch 73	Train 0.420532	Val 0.424237
Epoch 74	Train 0.376279	Val 0.404568
Epoch 75	Train 0.368342	Val 0.387985
Epoch 76	Train 0.361025	Val 0.392740
Epoch 77	Train 0.368124	Val 0.377808
Epoch 78	Train 0.358735	Val 0.374101
Epoch 79	Train 0.360838	Val 0.374007
Epoch 80	Train 0.351603	Val 0.370835
Epoch 81	Train 0.347138	Val 0.373368
Epoch 82	Train 0.348341	Val 0.370440
Epoch 83	Train 0.346185	Val 0.392960
Epoch 84	Train 0.339830	Val 0.360233
Epoch 85	Train 0.341650	Val 0.358871
Epoch 86	Train 0.327524	Val 0.355481
Epoch 87	Train 0.324445	Val 0.368723
Epoch 88	Train 0.327729	Val 0.352362
Epoch 89	Train 0.322427	Val 0.349424
Epoch 90	Train 0.316076	Val 0.360975
Epoch 91	Train 0.314550	Val 0.343328
Epoch 92	Train 0.312487	Val 0.342632
Epoch 93	Train 0.311315	Val 0.347997
Epoch 94	Train 0.312535	Val 0.346082
Epoch 95	Train 0.309983	Val 0.327836
Epoch 96	Train 0.314577	Val 0.333113
Epoch 97	Train 0.303207	Val 0.334618
Epoch 98	Train 0.306200	Val 0.342053
Epoch 99	Train 0.314484	Val 0.342983
Epoch 100	Train 0.306169	Val 0.325307

Testing RUN5...

===== RUN5 FINAL RESULTS =====

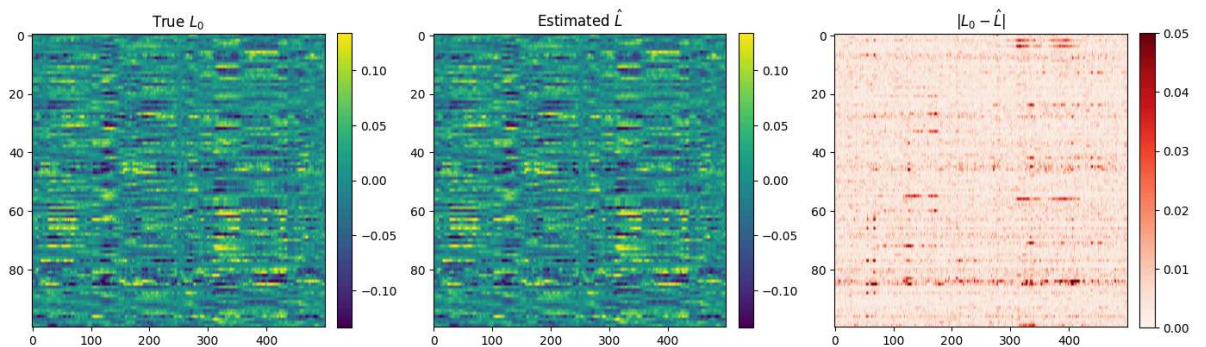
NMSE_L : 0.035608

NMSE_S : 0.282269

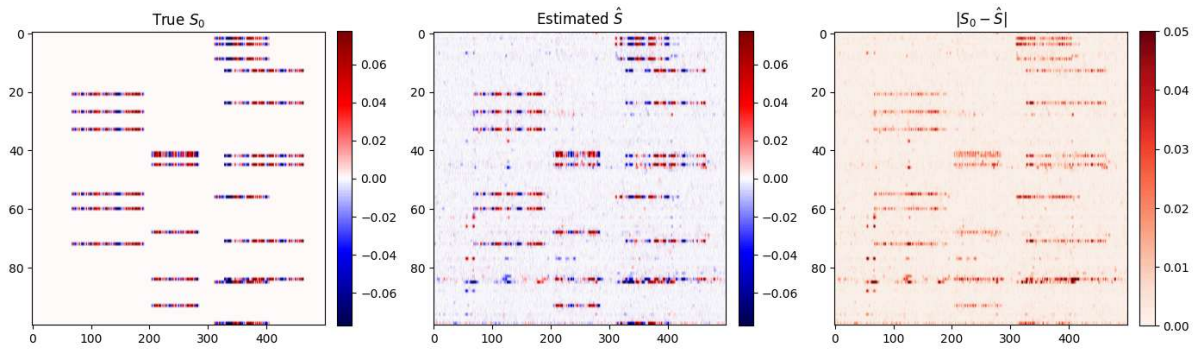
Layers : 10

=====

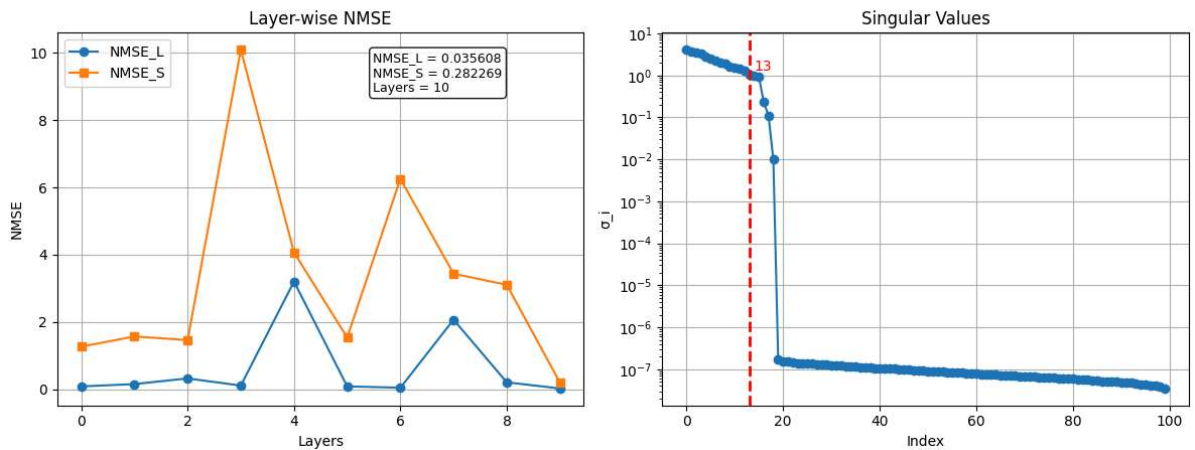
RUN5_UNFOLDED_ADMM_LEARNED_PROX — LOW RANK



RUN5_UNFOLDED_ADMM_LEARNED_PROX — SPARSE



RUN5_UNFOLDED_ADMM_LEARNED_PROX — DIAGNOSTICS



Saved structured figures for RUN5_UNFOLDED_ADMM_LEARNED_PROX

RUN5 COMPLETED.

RUN6 — Hybrid Residual ADMM

ADMM is unfolded into K layers as in the previous runs, but both the low-rank and sparse updates are augmented with learnable residual corrections.

Each layer performs:

- Standard model-based updates (SVT and soft-thresholding)
- Residual CNN corrections added to both L and S

This hybrid architecture preserves the ADMM structure while allowing data-driven residual learning to compensate for model mismatch.

It represents the most expressive model-based deep learning formulation in this study.

```
In [9]: # RUN6 — Hybrid Residual ADMM

print("\n=== RUN6 — Hybrid Residual ADMM ===")

# Residual CNN block

class ResidualCNN(nn.Module):
    def __init__(self, channels=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, channels, 3, padding=1),
            nn.ReLU(),
```



```

        nn.Conv2d(channels, channels, 3, padding=1),
        nn.ReLU(),
        nn.Conv2d(channels, 1, 3, padding=1)
    )
    self.alpha = nn.Parameter(torch.tensor(0.05))

def forward(self, x):
    return self.alpha * self.net(x)

# Hybrid unfolded ADMM

class DeepHybridADMM(nn.Module):
    def __init__(self, K, lambda_init, rho_init):
        super().__init__()
        self.K = K
        self.log_lambda = nn.Parameter(torch.log(torch.ones(K) * lambda_init))
        self.log_rho = nn.Parameter(torch.log(torch.ones(K) * rho_init))
        self.res_L = nn.ModuleList([ResidualCNN(32) for _ in range(K)])
        self.res_S = nn.ModuleList([ResidualCNN(32) for _ in range(K)])

    def forward(self, Y, L0=None, S0=None, return_curve=False):

        L = torch.zeros_like(Y)
        S = torch.zeros_like(Y)
        U = torch.zeros_like(Y)

        nmse_L_curve, nmse_S_curve = [], []

        for k in range(self.K):

            lambda_k = torch.exp(self.log_lambda[k])
            rho_k = torch.exp(self.log_rho[k])

            # L update (SVT + residual)
            L_svt = svt(Y - S + U, 1.0 / rho_k)
            L = L_svt + self.res_L[k](L_svt.unsqueeze(1)).squeeze(1)

            # S update (soft-threshold + residual)
            Z = Y - L + U
            S_soft = soft_threshold(Z, lambda_k / rho_k)
            S = S_soft + self.res_S[k](S_soft.unsqueeze(1)).squeeze(1)

            U = U + (Y - L - S)

            # Optional layer-wise NMSE tracking
            if return_curve and (L0 is not None) and (S0 is not None):
                nmse_L_curve.append(compute_nmse(L, L0).item())
                nmse_S_curve.append(compute_nmse(S, S0).item())

        if return_curve:
            return L, S, nmse_L_curve, nmse_S_curve
        return L, S

# Hyperparameters

K = 10
epochs = 100
batch_size = 4
lr = 5e-4
patience = 12

lambda_init = 1 / math.sqrt(max(N, T))

```

```

rho_init = 2.0

model = DeepHybridADMM(K, lambda_init, rho_init).to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

# Train / validation split

val_size = 10
train_idx = list(range(val_size, num_train))
val_idx = list(range(val_size))

best_val_loss = float("inf")
patience_counter = 0

print(f"Layers: {K}")
print("Training...\n")

# Training Loop

for epoch in range(epochs):

    model.train()
    train_loss = 0.0
    perm = torch.randperm(len(train_idx))

    for i in range(0, len(train_idx), batch_size):

        batch_ids = [train_idx[j.item()] for j in perm[i:i+batch_size]]

        Y = Y_train[batch_ids]
        L0 = L0_train[batch_ids]
        S0 = S0_train[batch_ids]

        L_hat, S_hat = model(Y)
        loss = compute_nmse(L_hat, L0) + compute_nmse(S_hat, S0)

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
        optimizer.step()

        train_loss += loss.item()

    train_loss /= (len(train_idx) / batch_size)

    # Validation

    model.eval()
    val_loss = 0.0

    with torch.no_grad():
        for i in val_idx:
            Y = Y_train[i:i+1]
            L0 = L0_train[i:i+1]
            S0 = S0_train[i:i+1]

            L_hat, S_hat = model(Y)
            val_loss += (compute_nmse(L_hat, L0) +
                        compute_nmse(S_hat, S0)).item()

    val_loss /= len(val_idx)

```

```

print(f"Epoch {epoch+1:3d} | Train {train_loss:.6f} | Val {val_loss:.6f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_state = model.state_dict()
    patience_counter = 0
else:
    patience_counter += 1
    if patience_counter >= patience:
        print("Early stopping triggered.")
        break

# Restore best model

model.load_state_dict(best_state)

# Test

print("\nTesting RUN6...\n")

model.eval()

with torch.no_grad():

    L_hat, S_hat = model(Y_test)
    nmse_L = compute_nmse(L_hat, L0_test).item()
    nmse_S = compute_nmse(S_hat, S0_test).item()

    _, _, nmse_L_curve, nmse_S_curve = model(
        Y_test[0:1],
        L0_test[0:1],
        S0_test[0:1],
        return_curve=True
    )

print("==== RUN6 FINAL RESULTS =====")
print(f"NMSE_L : {nmse_L:.6f}")
print(f"NMSE_S : {nmse_S:.6f}")
print(f"Layers : {K}")
print("=====")

# Save results

L_rep = L_hat[0].detach().cpu().numpy()
S_rep = S_hat[0].detach().cpu().numpy()
L0_rep = L0_test[0].detach().cpu().numpy()
S0_rep = S0_test[0].detach().cpu().numpy()

save_run_results(
    run_name="RUN6_HYBRID_RESIDUAL_ADMM",
    nmse_L_list=[nmse_L],
    nmse_S_list=[nmse_S],
    iterations_list=[K],
    L0_rep=L0_rep,
    L_hat_rep=L_rep,
    S0_rep=S0_rep,
    S_hat_rep=S_rep,
    convergence_curve=None,
    layerwise_nmse_L=nmse_L_curve,
    layerwise_nmse_S=nmse_S_curve,
    extra_params={

```

```
        "K": K,  
        "epochs": epoch+1,  
        "lambda_init": lambda_init,  
        "rho_init": rho_init  
    },  
    base_path=PROJECT_PATH  
)  
  
print("\nRUN6 COMPLETED.")
```

=== RUN6 – Hybrid Residual ADMM ===

Layers: 10

Training...

Epoch	1	Train	0.907698	Val	0.794811
Epoch	2	Train	0.743100	Val	0.697060
Epoch	3	Train	0.651813	Val	0.610686
Epoch	4	Train	0.577764	Val	0.541024
Epoch	5	Train	0.498941	Val	0.462335
Epoch	6	Train	0.432421	Val	0.416329
Epoch	7	Train	0.392092	Val	0.367872
Epoch	8	Train	0.349852	Val	0.340292
Epoch	9	Train	0.319448	Val	0.312407
Epoch	10	Train	0.296398	Val	0.291423
Epoch	11	Train	0.279940	Val	0.284492
Epoch	12	Train	0.267253	Val	0.269411
Epoch	13	Train	0.256975	Val	0.255475
Epoch	14	Train	0.245961	Val	0.248945
Epoch	15	Train	0.241353	Val	0.245713
Epoch	16	Train	0.233868	Val	0.242360
Epoch	17	Train	0.225336	Val	0.243349
Epoch	18	Train	0.226213	Val	0.226398
Epoch	19	Train	0.227253	Val	0.223810
Epoch	20	Train	0.212710	Val	0.217866
Epoch	21	Train	0.211982	Val	0.215925
Epoch	22	Train	0.205611	Val	0.214954
Epoch	23	Train	0.202570	Val	0.205432
Epoch	24	Train	0.197117	Val	0.202310
Epoch	25	Train	0.192514	Val	0.199805
Epoch	26	Train	0.198720	Val	0.198948
Epoch	27	Train	0.193033	Val	0.198001
Epoch	28	Train	0.192323	Val	0.193181
Epoch	29	Train	0.185461	Val	0.196335
Epoch	30	Train	0.182496	Val	0.198354
Epoch	31	Train	0.177730	Val	0.187409
Epoch	32	Train	0.178395	Val	0.189694
Epoch	33	Train	0.172890	Val	0.183957
Epoch	34	Train	0.177398	Val	0.183664
Epoch	35	Train	0.175465	Val	0.191361
Epoch	36	Train	0.169939	Val	0.180259
Epoch	37	Train	0.168048	Val	0.180127
Epoch	38	Train	0.169980	Val	0.175882
Epoch	39	Train	0.165157	Val	0.176467
Epoch	40	Train	0.164088	Val	0.178561
Epoch	41	Train	0.164638	Val	0.174092
Epoch	42	Train	0.160920	Val	0.170264
Epoch	43	Train	0.158808	Val	0.174675
Epoch	44	Train	0.158296	Val	0.179428
Epoch	45	Train	0.157333	Val	0.166715
Epoch	46	Train	0.153968	Val	0.171628
Epoch	47	Train	0.151890	Val	0.163626
Epoch	48	Train	0.155831	Val	0.165149
Epoch	49	Train	0.153078	Val	0.165554
Epoch	50	Train	0.152220	Val	0.172273
Epoch	51	Train	0.152644	Val	0.173808
Epoch	52	Train	0.150620	Val	0.166305
Epoch	53	Train	0.151955	Val	0.163933
Epoch	54	Train	0.147743	Val	0.167432
Epoch	55	Train	0.147143	Val	0.163804
Epoch	56	Train	0.142143	Val	0.158183
Epoch	57	Train	0.145054	Val	0.155192
Epoch	58	Train	0.143874	Val	0.159021
Epoch	59	Train	0.139788	Val	0.152139
Epoch	60	Train	0.140026	Val	0.153807

Epoch 61	Train 0.137798	Val 0.156359
Epoch 62	Train 0.139027	Val 0.156643
Epoch 63	Train 0.143207	Val 0.152894
Epoch 64	Train 0.138455	Val 0.161755
Epoch 65	Train 0.140147	Val 0.167870
Epoch 66	Train 0.139776	Val 0.152741
Epoch 67	Train 0.136231	Val 0.154556
Epoch 68	Train 0.133939	Val 0.153468
Epoch 69	Train 0.130306	Val 0.149188
Epoch 70	Train 0.130960	Val 0.149064
Epoch 71	Train 0.128535	Val 0.153548
Epoch 72	Train 0.132618	Val 0.151588
Epoch 73	Train 0.125775	Val 0.150709
Epoch 74	Train 0.128015	Val 0.153534
Epoch 75	Train 0.131169	Val 0.147872
Epoch 76	Train 0.131226	Val 0.146284
Epoch 77	Train 0.128115	Val 0.148010
Epoch 78	Train 0.125264	Val 0.145318
Epoch 79	Train 0.125979	Val 0.150318
Epoch 80	Train 0.125097	Val 0.134759
Epoch 81	Train 0.121324	Val 0.147762
Epoch 82	Train 0.122385	Val 0.146115
Epoch 83	Train 0.127753	Val 0.137278
Epoch 84	Train 0.125542	Val 0.140305
Epoch 85	Train 0.124793	Val 0.137233
Epoch 86	Train 0.121709	Val 0.141519
Epoch 87	Train 0.120121	Val 0.133388
Epoch 88	Train 0.119677	Val 0.136805
Epoch 89	Train 0.117293	Val 0.134932
Epoch 90	Train 0.119138	Val 0.134830
Epoch 91	Train 0.121431	Val 0.135838
Epoch 92	Train 0.114028	Val 0.128925
Epoch 93	Train 0.113868	Val 0.131095
Epoch 94	Train 0.113877	Val 0.131066
Epoch 95	Train 0.115767	Val 0.135481
Epoch 96	Train 0.118016	Val 0.137584
Epoch 97	Train 0.115157	Val 0.132081
Epoch 98	Train 0.117238	Val 0.136370
Epoch 99	Train 0.113092	Val 0.126503
Epoch 100	Train 0.112691	Val 0.134474

Testing RUN6...

===== RUN6 FINAL RESULTS =====

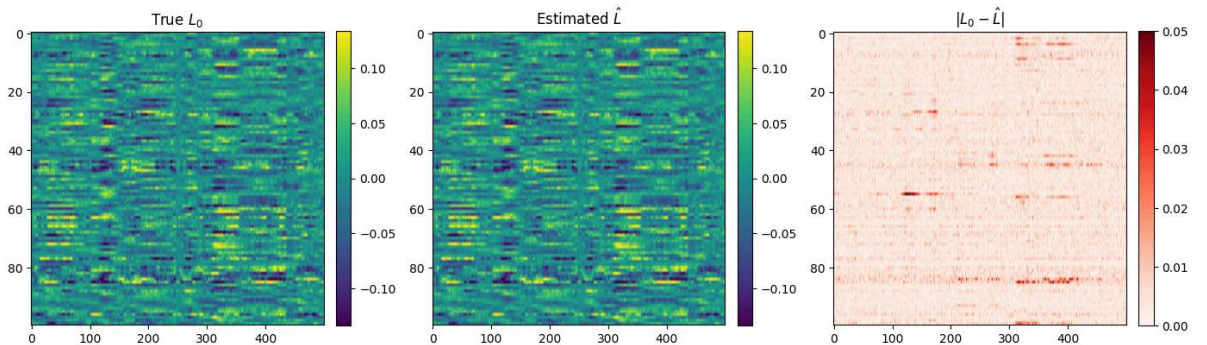
NMSE_L : 0.019118

NMSE_S : 0.120928

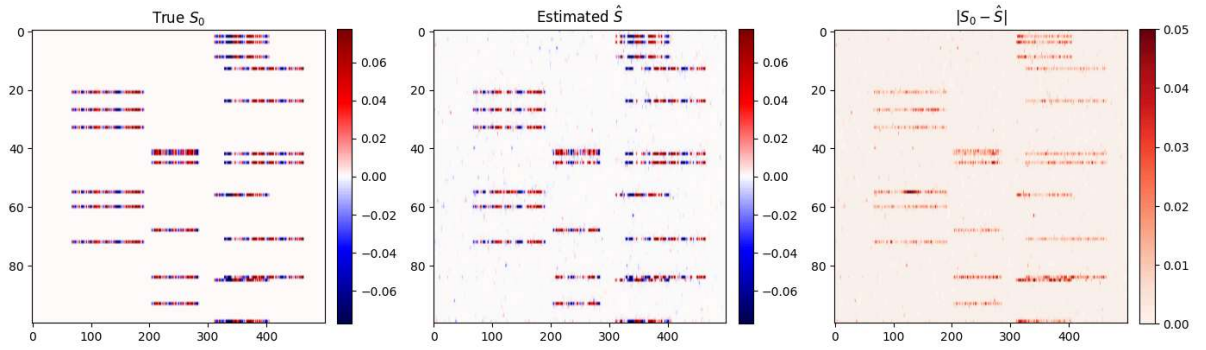
Layers : 10

=====

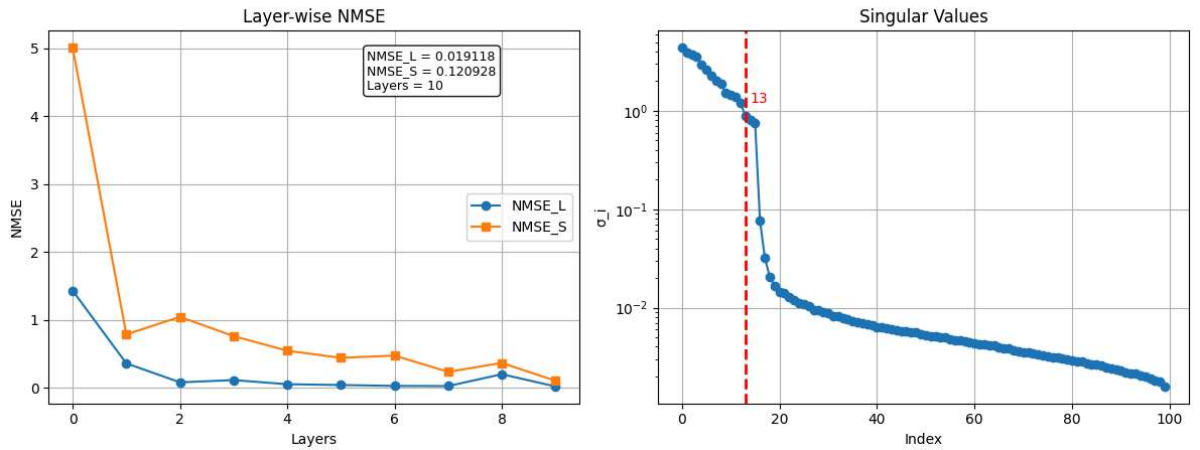
RUN6_HYBRID_RESIDUAL_ADMM — LOW RANK



RUN6_HYBRID_RESIDUAL_ADMM — SPARSE



RUN6_HYBRID_RESIDUAL_ADMM — DIAGNOSTICS



Saved structured figures for RUN6_HYBRID_RESIDUAL_ADMM

RUN6 COMPLETED.

Final Comparison — Mismatch Dataset

This section summarizes the performance of all approaches under model mismatch conditions:

- **NMSE_L** and **NMSE_S** (log-scale) to compare reconstruction accuracy.
- **Efficiency vs Accuracy**, showing the trade-off between computational complexity (iterations/layers) and total reconstruction error.
- **Relative improvement** over classical ADMM, highlighting the benefit of model-based deep learning extensions.

The comparison illustrates the progressive transition from pure model-based optimization to hybrid residual architectures, quantifying how learning improves robustness under mismatch.

```
In [15]: # FINAL COMPARISON — Summary (Mismatch Dataset)

print("\n=== FINAL COMPARISON ===")

metrics_path = os.path.join(PROJECT_PATH, "results_metrics")
figures_path = os.path.join(PROJECT_PATH, "results_figures")

runs = [
    "RUN2_ADMM_BASELINE_MISMATCH",
    "RUN3_GLOBAL_LEARNED_ADMM",
    "RUN4_LAYERWISE_UNFOLDED_ADMM",
    "RUN5_UNFOLDED_ADMM_LEARNED_PROX",
```



```

"RUN6_HYBRID_RESIDUAL_ADMM"
]

labels = [
    "ADMM (Baseline Mismatch)",
    "Global-Learned ADMM",
    "Layer-wise Unfolded ADMM",
    "Unfolded ADMM + Learned Prox",
    "Hybrid DL Residual ADMM"
]

nmse_L, nmse_S, complexity = [], [], []

for run in runs:
    data = np.load(os.path.join(metrics_path, f"{run}_metrics.npz"))
    nmse_L.append(data["NMSE_L_mean"])
    nmse_S.append(data["NMSE_S_mean"])
    complexity.append(data["Iterations_mean"])

nmse_L = np.array(nmse_L)
nmse_S = np.array(nmse_S)
complexity = np.array(complexity)

nmse_total = nmse_L + nmse_S

# Colors (model-based → hybrid progression)

colors = [
    "#1f77b4", # ADMM
    "#1f77b4", # Global Learned
    "#ff7f0e", # Unfolded
    "#ff7f0e", # Unfolded + prox
    "#d62728"  # Hybrid
]

# Create 2x2 figure

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("Global Comparison – Mismatch Dataset", fontsize=16)

x = np.arange(len(labels))

# 1) NMSE_L

ax = axes[0, 0]
ax.bar(x, nmse_L, color=colors)
ax.set_yscale("log")
ax.set_title("NMSE_L (Low-Rank)")
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30)
ax.grid(True, which="both", linestyle="--", alpha=0.4)

for i, v in enumerate(nmse_L):
    ax.text(i, v * 1.05, f"{v:.3f}", ha="center", fontsize=9)

# 2) NMSE_S

ax = axes[0, 1]
ax.bar(x, nmse_S, color=colors)
ax.set_yscale("log")
ax.set_title("NMSE_S (Sparse)")
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30)
ax.grid(True, which="both", linestyle="--", alpha=0.4)

```

```

for i, v in enumerate(nmse_S):
    ax.text(i, v * 1.05, f"{v:.3f}", ha="center", fontsize=9)

# 3) Efficiency vs Accuracy

ax = axes[1, 0]
ax.scatter(complexity, nmse_total, s=120, c=colors)

for i, txt in enumerate(labels):
    ax.annotate(
        txt,
        (complexity[i], nmse_total[i]),
        textcoords="offset points",
        xytext=(6, 6),
        fontsize=9
    )

ax.set_xlabel("Iterations / Layers")
ax.set_ylabel("NMSE_L + NMSE_S")
ax.set_title("Efficiency vs Accuracy")
ax.grid(True)

# 4) Relative improvement over baseline ADMM

baseline_total = nmse_total[0]
impr_total = (baseline_total - nmse_total) / baseline_total * 100

ax = axes[1, 1]
ax.bar(x, impr_total, color=colors)
ax.axhline(0, color="black", linewidth=1)
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=30)
ax.set_ylabel("Improvement over ADMM (%)")
ax.set_title("Relative Improvement")
ax.grid(True, linestyle="--", alpha=0.4)

for i, v in enumerate(impr_total):
    ax.text(i, v + np.sign(v) * 0.5, f"{v:.1f}%", ha="center", fontsize=9)

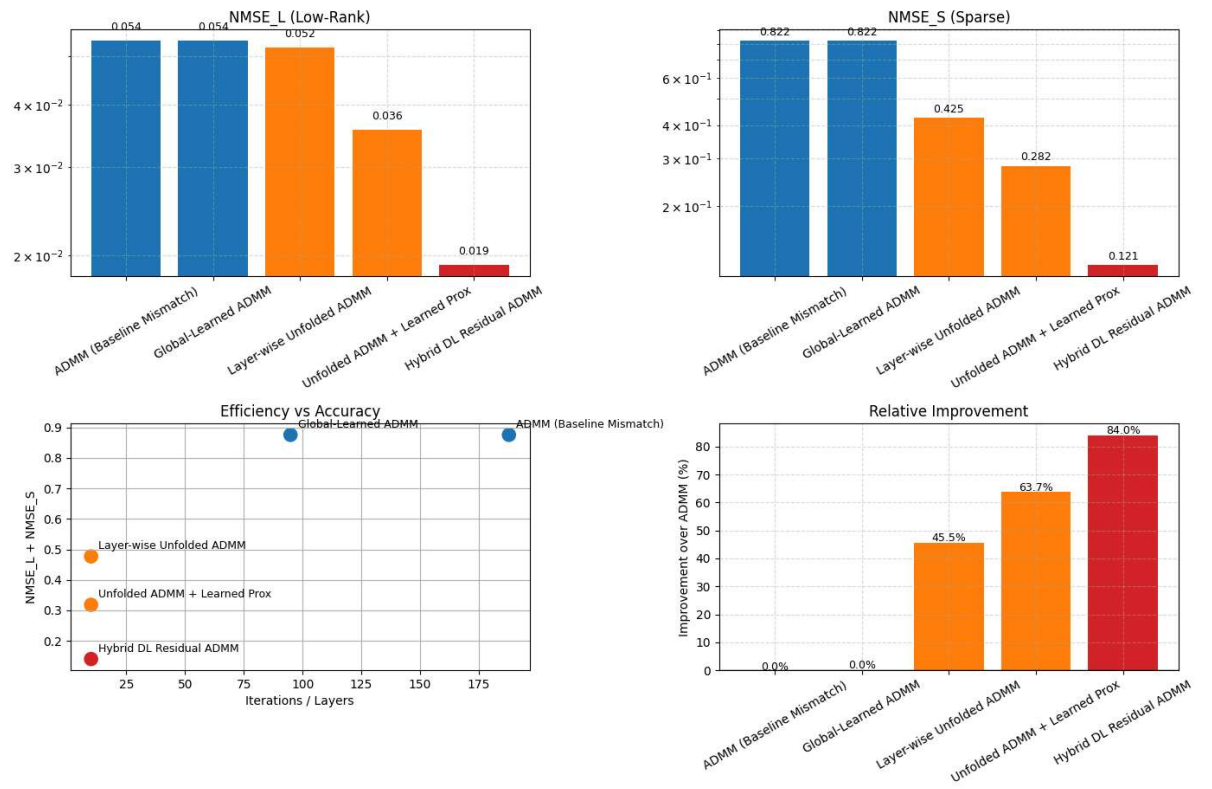
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig(os.path.join(figures_path, "FINAL_2x2_SUMMARY.png"), dpi=300)
plt.show()

print("\n2x2 summary figure saved.")

```

=== FINAL COMPARISON ===

Global Comparison — Mismatch Dataset



2x2 summary figure saved.